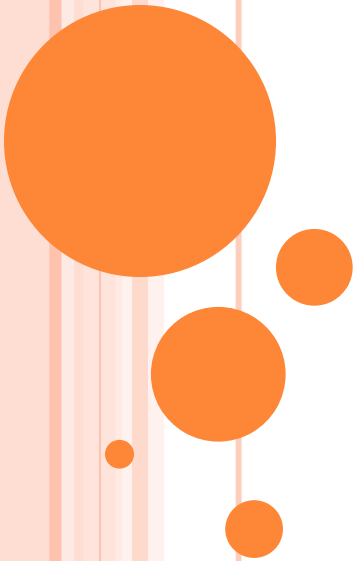


STRUCTURED QUERY LANGUAGE - JOIN

- SQL JOIN clause is used to query and access data from multiple tables by establishing logical relationships between them. It can access data from multiple tables simultaneously using common key values shared across different tables.
- Example of SQL JOINS
  - Consider the two tables, Student and StudentCourse, which share a common column ROLL_NO. Using SQL JOINS, we can combine data from these tables based on their relationship, allowing us to retrieve meaningful information like student details along with their enrolled courses.



- Student

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	HARSH	DELHI	xxxxxxxxxx	18
2	PRATIK	BIHAR	xxxxxxxxxx	19
3	RIYANKA	SILIGURI	xxxxxxxxxx	20
4	DEEP	RAMNAGAR	xxxxxxxxxx	18
5	SAPTARHI	KOLKATA	xxxxxxxxxx	19
6	DHANRAJ	BARABAJAR	xxxxxxxxxx	20
7	ROHIT	BALURGHAT	xxxxxxxxxx	18
8	NIRAJ	ALIPUR	xxxxxxxxxx	19

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

Both these tables are connected by one common key (column) i.e **ROLL_NO**. We can perform a JOIN operation using the given SQL query:

```
SELECT s.roll_no, s.name, s.address, s.phone, s.age, sc.course_id
FROM Student s
JOIN StudentCourse sc ON s.roll_no = sc.roll_no;
```

ROLL_NO	NAME	ADDRESS	PHONE	AGE	COURSE_ID
1	HARSH	DELHI	XXXXXXXXXX	18	1
2	PRATIK	BIHAR	XXXXXXXXXX	19	2
3	RIYANKA	SILGURI	XXXXXXXXXX	20	2
4	DEEP	RAMNAGAR	XXXXXXXXXX	18	3
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19	1

JOINED RELATIONS

- **Join operations** take two relations and return as a result another relation.
- These additional operations are typically used as subquery expressions in the **from** clause
- **Join condition** – defines which tuples in the two relations match, and what attributes are present in the result of the join.
- **Join type** – defines how tuples in each relation that do not match any tuple in the other relation (based on the join condition) are treated.



TRADITIONAL METHOD OF JOINING

- In SQL, the traditional method of joining tables refers to using the WHERE clause instead of the modern JOIN syntax. This approach was commonly used before explicit JOIN keywords became widely adopted in SQL-92.

Example of Traditional SQL Join (**Using WHERE Clause**)

```
SELECT employees.name, departments.department_name
FROM employees, departments
WHERE employees.department_id =
departments.department_id;
```

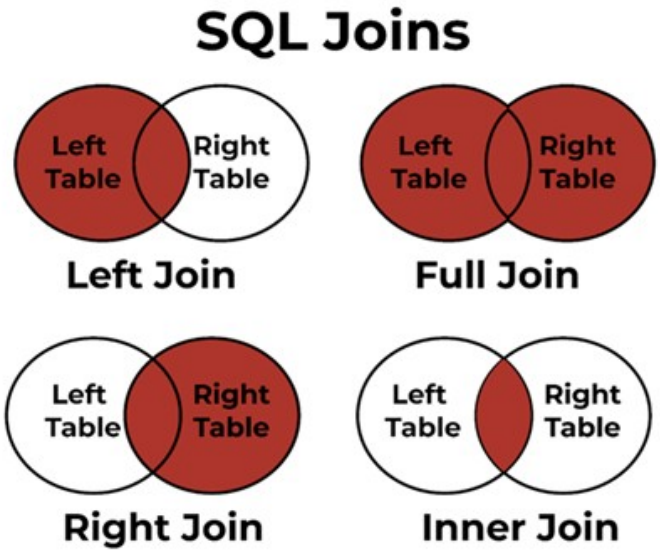
- Uses the WHERE Clause – The condition for joining tables is written in the WHERE clause.
- Comma-Separated Table List – Tables are separated by commas in the FROM clause.



JOIN TYPES

Join types
inner join
left outer join
right outer join
full outer join

Join Conditions
natural
on <predicate>
using $(A_1, A_1, \ldots, A_n)$



INNER JOIN

- Retrieves records that have matching values in both tables.
- If there is no match, the row is excluded.

```
SELECT * FROM table1 INNER JOIN table2
ON table1.column_name =
table2.column_name;
```

Eg:

```
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments ON employees.department_id =
departments.department_id;
```



NATURAL JOIN IN SQL

- A **NATURAL JOIN** is a type of SQL join that automatically matches columns with the same name in both tables and returns only the rows where those columns have equal values.

```
SELECT *
FROM table1
NATURAL JOIN table2;
```

```
SELECT * FROM
employees NATURAL
JOIN departments;
```

Key Features of NATURAL JOIN:

- It automatically matches common column(s) between tables.
- No need to specify the join condition (ON table1.column = table2.column).
- Only returns rows where the matched columns have the same values.
- If there are no common columns, it results in a Cartesian product (like a CROSS JOIN).

Caution with NATURAL JOIN:

•If column names do not match, the join may produce unexpected results.
•If multiple columns match, all of them are used for the join, which might lead to unintended filtering.

OUTER JOINS-DATASET

- Relation *loan*
- Relation *borrower*

loan_number	*branch_name*	*amount*
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

loan

customer_name	*loan_number*
Jones	L-170
Smith	L-230
Hayes	L-155

borrower

- Note: borrower information missing for L-260 and loan information missing for L-155



LEFT OUTER JOIN– EXAMPLES

- Retrieves all records from the left table (table1) and the matched records from the right table (table2).
- If no match is found, NULL values are returned for columns from the right table.

- *loan* **left outer join** *borrower* **on** *loan.loan_number = borrower.loan_number*

loan_number	*branch_name*	*amount*	*customer_name*	*loan_number*
L-170	Downtown	3000	Jones	L-170
L-230	Redwood	4000	Smith	L-230
L-260	Perryridge	1700	*null*	*null*

```
SELECT *
FROM table1
LEFT JOIN table2 ON table1.column_name =
table2.column_name;
```

RIGHT OUTER JOIN– EXAMPLES

- Retrieves all records from the right table (table2) and the matched records from the left table (table1).
- If no match is found, NULL values are returned for columns from the left table.

- *loan* **right outer join** *borrower* **on** *loan.loan_number = borrower.loan_number*

loan_number	*branch_name*	*amount*	*customer_name*
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-155	*null*	*null*	Hayes

```
SELECT *
FROM table1
RIGHT JOIN table2 ON table1.column_name =
table2.column_name;
```

FULL OUTER JOIN– EXAMPLES

- *loan* **full outer join** *borrower* **using** (*loan_number*)

- Retrieves records when there is a match in either left (table1) or right (table2) table.
- If there is no match, NULL values are returned for columns from the table without a match.

loan_number	*branch_name*	*amount*	*customer_name*
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	*null*
L-155	*null*	*null*	Hayes

```
SELECT *
FROM table1
FULL JOIN table2 ON table1.column_name =
table2.column_name;
```

- Find all customers who have either an account or a loan (but not both) at the bank.

select *customer_name*
from (*depositor* **natural full outer join** *borrower*)
where *account_number* **is null or** *loan_number* **is null**